

```
In [1]: import numpy as np
```

```
In [2]: # Create an array from a List
a=np.array([1,2,3])
print("array a:",a)
```

array a: [1 2 3]

```
In [3]: # Create an array with evenly spaced values
b=np.arange(0,10,3)
print("Array b:",b)
```

Array b: [0 3 6 9]

```
In [4]: d=np.zeros((2,3))
print("array d:\n",d)
```

array d:
[[0. 0. 0.]
 [0. 0. 0.]]

```
In [7]: # Create an array filled with ones
e=np.ones((3,2))
print("array e:\n",e)
```

array e:
[[1. 1.]
 [1. 1.]
 [1. 1.]]

```
In [8]: # Create an identity matrix
f=np.eye(4)
print("identity matrix f:\n",f)
```

identity matrix f:
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]

```
In [10]: #array manipulation functions
#Reshape an array
a1=np.array([1,2,3])
reshaped=np.reshape(a1,(1,3))
print("reshaped array:",reshaped)
```

reshaped array: [[1 2 3]]

```
In [13]: #Flatten an array
f1=np.array([[1,2],[3,4],[5,6],[7,8],[9,10]])
flattened=np.ravel(f1)
print("flattened array:",flattened)
```

flattened array: [1 2 3 4 5 6 7 8 9 10]

```
In [14]: # Transpose an array
e1=np.array([[1,2],[3,4]])
transposed=np.transpose(e1)
print("transposed array:\n",transposed)
```

transposed array:

```
[[1 3]
 [2 4]]
```

```
In [15]: # Stack arrays vertically
a2=np.array([1,2])
b2=np.array([3,4])
stacked=np.vstack([a2,b2])
print("stacked arrays:\n",stacked)
```

stacked arrays:

```
[[1 2]
 [3 4]]
```

```
In [16]: # Stack arrays horizontal
a2 = np.array([1, 2])
b2 = np.array([3, 4])
stacked = np.hstack([a2, b2]) # Stack a and b vertically
print("Stacked arrays:\n", stacked)
```

Stacked arrays:

```
[1 2 3 4]
```

```
In [17]: #mathematical functiuons
g=np.array([1,2,3,4])
added=np.add(g,2)
print("added 2 to g:",added)
```

added 2 to g: [3 4 5 6]

```
In [18]: # Square each element
squared=np.power(g,2)
print("squared g:",squared)
```

squared g: [1 4 9 16]

```
In [20]: #dot product
a3=np.array([1,2,3])
dot_product=np.dot(a1,a)
print("dot product of a1 and a",dot_product)
```

dot product of a1 and a 14

```
In [21]: a
```

```
Out[21]: array([1, 2, 3])
```

```
In [22]: #statistical functions
s=np.array([1,2,3,4])
mean=np.mean(s)
print("mean of s:",mean)
```

mean of s: 2.5

```
In [23]: var=np.var(s)
print("mean of s:",var)
```

mean of s: 1.25

```
In [25]: # Minimum element of an array
minimum=np.min(s)
print("min of s:",minimum)
```

min of s: 1

```
In [26]: # Maximum element of an array
maximum=np.max(s)
print("max of s",max)
```

max of s <built-in function max>

```
In [27]: #random sampling functions
# Generate random values between 0 and 1
random_vals=np.random.rand(3)
print("random values:",random_vals)
```

random values: [0.01033789 0.23168627 0.7850296]

```
In [35]: # Set seed for reproducibility
np.random.seed(0)
arr=np.random.randint(1,100,5)
print(arr)
```

[71 89 89 13 59]

```
In [39]: #boolean and logical functions
# Check if all elements are True
logical_test=np.array([False,False,False])
all_true=np.all(logical_test)
print("all elements true:",all_true)
```

all elements true: False

```
In [42]: #set operations
set_a=np.array([1,2,3,4])
set_b=np.array([3,4,5,6])
intersection=np.intersect1d(set_a,set_b)
print("intersection of a and b:",intersection)
```

intersection of a and b: [3 4]

```
In [44]: #array attribute functions
a=np.array([1,2,3])
shape=a.shape#shape of the array
size=a.size # no of elements
dimensions=a.ndim # number of dimensions
dtype=a.dtype #datatype of array
print("Shape of a:", shape)
print("Size of a:", size)
print("Number of dimensions of a:", dimensions)
print("Data type of a:", dtype)
```

Shape of a: (3,)

Size of a: 3

Number of dimensions of a: 1

Data type of a: int64

```
In [45]: #other functions
#Create a copy of an array
a = np.array([1, 2, 3])
copied_array = np.copy(a) # Create a copy of array a
print("Copied array:", copied_array)
```

Copied array: [1 2 3]

```
In [46]: #size in bytes of an array  
array_size=a.nbytes  
print("size of a in bytes:",array_size)
```

size of a in bytes: 24

```
In [47]: # Check if two arrays share memory  
shared=np.shares_memory(a,copied_array)  
print("do they share memory?",shared)
```

do they share memory? False